

DELETE vs TRUNCATE vs DROP in SQL

FEATURE	DELETE	TRUNCATE	DROP
Removes	Specific rows from a table	All rows from a table	Entire table
Command Type	DML	DDL	DDL
WHERE Clause	Can use WHERE	Cannot use WHERE	Cannot use WHERE
Rollback	Can be rolled back	Cannot be rolled back*	Cannot be rolled back*
Trigger	DELETE trigger can fire	Usually does not fire DELETE trigger	No DELETE trigger
Table Structure	Remains	Remains	Removed
Performance	Slower	Faster	Fast

WHERE vs HAVING in SQL

FEATURE	WHERE	HAVING
Purpose	Filters individual rows	Filters groups
Applied	Before <code>GROUP BY</code>	After <code>GROUP BY</code>
Filtering Level	Row-level filtering	Group-level filtering
Aggregate Functions	Cannot directly use aggregate functions	Can use aggregate functions
<code>COUNT ()</code>	Cannot directly filter using <code>COUNT ()</code>	Can filter using <code>COUNT ()</code>
<code>SUM ()</code>	Cannot directly filter using <code>SUM ()</code>	Can filter using <code>SUM ()</code>
<code>AVG ()</code>	Cannot directly filter using <code>AVG ()</code>	Can filter using <code>AVG ()</code>
Used Without <code>GROUP BY</code>	Yes	Database-dependent
Example	<code>WHERE salary > 50000</code>	<code>HAVING COUNT (*) > 5</code>

UNION vs UNION ALL in SQL

FEATURE	UNION	UNION ALL
Purpose	Combines result sets	Combines result sets
Duplicate Records	Removes duplicates	Keeps duplicates
Performance	Slower	Faster
Duplicate Elimination	Yes	No
Sorting/Processing	Additional processing may be required	Less processing
Number of Columns	Must be same	Must be same
Column Data Types	Must be compatible	Must be compatible
Best Use	When duplicate removal is required	When all records are required

**INNER JOIN vs LEFT JOIN vs RIGHT JOIN
vs FULL JOIN****

FEATURE	INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL JOIN
Matching Rows	Yes	Yes	Yes	Yes
All Left Rows	No	Yes	No	Yes
All Right Rows	No	No	Yes	Yes
Unmatched Left Rows	Excluded	Included	Excluded	Included
Unmatched Right Rows	Excluded	Included as NULL	Included	Included
Main Purpose	Matching records	Keep all left records	Keep all right records	Keep all records
NULL Values	Usually fewer	Possible	Possible	Possible

JOIN vs UNION in SQL

FEATURE	JOIN	UNION
Combines	Columns	Rows

<i>FEATURE</i>	<i>JOIN</i>	<i>UNION</i>
Direction	Horizontal	Vertical
Requirement	Join condition usually required	Compatible SELECT statements
Number of Columns	Can be different between source tables	Must match
Matching	Based on related columns	Appends result sets
Example	Customer + Order	Customers + Employees
Main Purpose	Combine related data	Combine similar datasets

PRIMARY KEY vs UNIQUE KEY in SQL

<i>FEATURE</i>	<i>PRIMARY KEY</i>	<i>UNIQUE KEY</i>
Purpose	Uniquely identifies each row	Ensures unique values
Duplicate Values	Not allowed	Not allowed

FEATURE	PRIMARY KEY	UNIQUE KEY
NULL Values	Not allowed	Database-dependent
Number per Table	One primary key constraint	Multiple unique constraints possible
Foreign Key Reference	Commonly referenced	Can be referenced depending on database
Main Usage	Main row identifier	Alternate unique identifier
Index	Usually creates supporting index	Usually creates supporting index

PRIMARY KEY vs FOREIGN KEY in SQL

FEATURE	PRIMARY KEY	FOREIGN KEY
Purpose	Identifies a row uniquely	Creates relationship between tables
Duplicate Values	Not allowed	Allowed
NULL Values	Not allowed	Can be allowed

FEATURE	PRIMARY KEY	FOREIGN KEY
Uniqueness	Must be unique	Does not need to be unique
Table Relationship	Parent identifier	Child reference
Number per Table	One primary key constraint	Multiple foreign keys possible
Example	<code>customer_id</code>	<code>customer_id</code> in orders

GROUP BY vs ORDER BY in SQL

FEATURE	GROUP BY	ORDER BY
Purpose	Groups rows	Sorts rows
Aggregation	Commonly used with aggregates	Not required
Changes Number of Rows	Can reduce rows with aggregation	Normally does not reduce rows
Sorting	Does not guarantee final sorting	Explicitly sorts result

FEATURE	GROUP BY	ORDER BY
Common Functions	COUNT () , SUM () , AVG ()	ASC , DESC
Position	Before ORDER BY	Usually near end
Example	GROUP BY department	ORDER BY salary DESC

WHERE vs ON in SQL

FEATURE	WHERE	ON
Purpose	Filters records	Defines JOIN matching condition
Main Usage	General filtering	JOIN condition
Applied	After JOIN logically	During JOIN matching
LEFT JOIN Impact	Can remove unmatched rows	Preserves unmatched left rows
Typical Condition	WHERE salary > 50000	ON c.id = o.customer_id

FEATURE	WHERE	ON
Used Without JOIN	Yes	Usually associated with JOIN
Main Focus	Filter result	Connect tables

EXISTS vs IN in SQL

FEATURE	EXISTS	IN
Purpose	Checks whether matching rows exist	Checks whether value exists in a set
Result	TRUE/FALSE condition	Value comparison
Correlated Query	Commonly used	Less commonly correlated
Processing	Can stop after finding a match	Depends on query/database optimizer
NULL Behavior	Generally easier to reason about	Can have surprising NULL behavior with <code>NOT IN</code>
Best Use	Checking existence	Comparing against a list/set

FEATURE	EXISTS	IN
Example	WHERE EXISTS (...)	WHERE id IN (...)

IN vs BETWEEN in SQL

FEATURE	IN	BETWEEN
Purpose	Matches specific values	Matches a range
Values	Discrete values	Range of values
Example	IN (10, 20, 30)	BETWEEN 10 AND 30
Use Case	Specific options	Numeric/date ranges
Multiple Values	Yes	No, uses lower and upper bounds
Range Support	No	Yes
Boundary Values	Exact values only	Usually inclusive

CTE vs SUBQUERY in SQL

FEATURE	CTE	SUBQUERY
Syntax	Uses <code>WITH</code>	Uses nested <code>SELECT</code>
Readability	Usually better for complex queries	Can become difficult when deeply nested
Multiple Queries	Multiple CTEs can be defined	Multiple nested subqueries possible
Recursive Query	Supported	Not directly
Scope	Statement	Query/expression
Reusability	Can reference a CTE multiple times within the statement	Usually repeated if needed
Best Use	Complex multi-step logic	Simple nested logic

CTE vs TEMP TABLE in SQL

FEATURE	CTE	TEMP TABLE
Type	Query expression	Temporary database object
Storage	No guaranteed physical storage	Stores data
Scope	Usually one statement	Session/transaction dependent
Index	Cannot normally create an index on CTE	Can often create indexes
Reuse	Within the statement	Across multiple statements
Best Use	Query organization	Multi-step processing
Persistence	Temporary logical result	Temporary physical object

RANK vs DENSE_RANK vs ROW_NUMBER in SQL

FEATURE	ROW_NUMBER	RANK	DENSE_RANK
Duplicate Ranking	Gives different numbers	Same rank for ties	Same rank for ties

FEATURE	ROW_NUMBER	RANK	DENSE_RANK
Ranking Gaps	No	Yes	No
Example for 100,100,90	1,2,3	1,1,3	1,1,2
Tie Handling	No tie	Supports ties	Supports ties
Unique Sequence	Yes	No	No
Common Use	Select one row per group	Competition ranking	Ranking without gaps

COUNT() vs COUNT(column) vs COUNT(DISTINCT column)***

FEATURE	COUNT(*)	COUNT(COLUMN)	COUNT(DISTINCT COLUMN)
Counts	All rows	Non-NULL values	Unique non-NULL values
Counts NULL	Yes	No	No
Removes Duplicates	No	No	Yes

FEATURE	COUNT(*)	COUNT(COLUMN)	COUNT(DISTINCT COLUMN)
Typical Usage	Total records	Records with a value	Unique values
Example	COUNT (*)	COUNT (email)	COUNT (DISTINCT city)

VIEW vs MATERIALIZED VIEW in SQL

FEATURE	VIEW	MATERIALIZED VIEW
Stores Query Definition	Yes	Yes
Stores Query Result	Normally no	Yes
Data Freshness	Reflects source data when queried	Can become stale
Refresh Required	No	Yes
Storage Required	Minimal	Requires storage
Query Performance	Depends on underlying query	Often faster for expensive aggregations

FEATURE	VIEW	MATERIALIZED VIEW
Best Use	Logical abstraction	Precomputed analytical results

CHAR vs VARCHAR in SQL

FEATURE	CHAR	VARCHAR
Length	Fixed	Variable
Storage	Fixed-length behavior	Variable-length behavior
Best For	Fixed-size values	Variable-size strings
Example	Country code	Customer name
Padding	May pad values depending on database	Generally no fixed-length padding
Flexibility	Lower	Higher

DECIMAL vs FLOAT in SQL

FEATURE	DECIMAL	FLOAT
Precision	Exact decimal arithmetic	Approximate
Best For	Financial data	Scientific calculations
Rounding Issues	More predictable	Possible
Storage	Depends on precision/database	Typically compact for approximate values
Example	<code>DECIMAL(18,2)</code>	<code>FLOAT</code>
Money Usage	Recommended	Usually avoided

NULL vs 0 vs EMPTY STRING in SQL

FEATURE	NULL	0	EMPTY STRING
Meaning	Missing/unknown value	Numeric zero	Empty text value

FEATURE	NULL	0	EMPTY STRING
Data Type	Can occur across nullable types	Numeric	Character/string
Comparison	IS NULL	= 0	= ''
Represents Nothing	Unknown/missing	A numeric value	A zero-length string
Example	phone IS NULL	salary = 0	name = ''

OLTP vs OLAP**

FEATURE	OLTP	OLAP
Full Form	Online Transaction Processing	Online Analytical Processing
Purpose	Transaction processing	Data analysis
Operations	INSERT, UPDATE, DELETE	Mostly SELECT
Query Type	Simple and short	Complex and analytical

FEATURE	OLTP	OLAP
Data	Current operational data	Historical/analytical data
Schema	Often normalized	Often denormalized
Users	Applications/end users	Analysts/data teams
Performance Goal	Fast transactions	Fast analytical queries
Data Volume	Usually smaller transactions	Large datasets
Example	Banking transaction system	Data warehouse

DELETE vs UPDATE in SQL

FEATURE	DELETE	UPDATE
Purpose	Removes rows	Modifies existing rows
Data	Removed	Retained with changed values

FEATURE	DELETE	UPDATE
WHERE Clause	Supported	Supported
SET Clause	Not applicable	Required for column changes
Rollback	Depends on transaction/database	Depends on transaction/database
Example	<pre>DELETE FROM employee WHERE id = 10</pre>	<pre>UPDATE employee SET salary = 50000 WHERE id = 10</pre>

STORED PROCEDURE vs FUNCTION in SQL

FEATURE	STORED PROCEDURE	FUNCTION
Purpose	Performs operations/workflows	Performs reusable computation
Return Value	May return values/result sets depending on DB	Designed to return a value/result
Usage in SELECT	Database-dependent	Often usable in SQL expressions
Parameters	Supported	Supported

<i>FEATURE</i>	<i>STORED PROCEDURE</i>	<i>FUNCTION</i>
DML	Database-dependent	Database-dependent
Best Use	Business workflows	Reusable calculations
Invocation	Procedure-specific syntax	Function call syntax

INDEX SCAN vs SEQUENTIAL SCAN in PostgreSQL

<i>FEATURE</i>	<i>INDEX SCAN</i>	<i>SEQUENTIAL SCAN</i>
Access Method	Uses index	Reads table sequentially
Best For	Selective queries	Large percentage of table
Random I/O	Can involve random reads	Mostly sequential reads
Index Required	Yes	No
Small Result Set	Often efficient	May be inefficient

FEATURE	INDEX SCAN	SEQUENTIAL SCAN
Large Result Set	May be less efficient	Often efficient
Decision	Cost-based optimizer	Cost-based optimizer

PRIMARY KEY vs INDEX in SQL

FEATURE	PRIMARY KEY	INDEX
Purpose	Enforces row identity	Improves data retrieval
Uniqueness	Yes	Optional
NULL	Not allowed	Depends on index/database
Constraint	Yes	No
Multiple Allowed	One PK constraint	Multiple indexes
Performance	Provides supporting index in many DBs	Designed for query performance

FEATURE	PRIMARY KEY	INDEX
Foreign Key	Can be referenced	Not automatically a relationship

NORMALIZATION vs DENORMALIZATION**

FEATURE	NORMALIZATION	DENORMALIZATION
Main Goal	Reduce redundancy	Improve read performance
Duplicate Data	Minimized	May be intentionally introduced
JOINS	More likely	Often reduced
Write Performance	Can be more complex	Can be simpler
Read Performance	Can require more joins	Often faster for reads
Data Integrity	Generally easier to maintain	Requires more care
Common Usage	OLTP	OLAP / reporting